

1. Data Preparation

Data preparation is the process of converting raw data into a format suitable for machine learning.

In real life, data is often:

- Incomplete
- Noisy
- Duplicated
- Inconsistent

A machine learning model can only learn effectively if the data is clean and organized.

Why is Data Preparation Important?

Suppose you want to predict student marks.

Study Hours	Marks
2	50
3	65
?	70
5	Missing

This dataset contains missing values. If we train a model directly, the results may be incorrect.

Data preparation helps:

- Improve model accuracy
- Reduce errors
- Increase training efficiency
- Avoid misleading predictions

Train-Test Split

Train-Test Split is the process of dividing a dataset into two parts:

Training Set

Used to teach the machine learning model.

Testing Set

Used to evaluate how well the model learned.

Common split ratios:

Training	Testing
80%	20%
70%	30%
90%	10%

Why Do We Need It?

Imagine a student who memorizes all answers from a book.

If you ask the same questions, the student scores 100%.

But if you ask new questions, the score may be low.

Similarly, a machine learning model must be tested on unseen data.

Example

Suppose a dataset contains 1000 records.

- 800 records → Training
- 200 records → Testing

The model learns from the 800 records and is evaluated on the 200 records.

This helps determine whether the model can generalize to new data.

CODE:-

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
print("Training Data:", len(X_train))
```

```
print("Testing Data:", len(X_test))
```

Training Data: 120

Testing Data: 30

Validation Techniques

Validation techniques are used to evaluate model performance before final deployment.

The goal is to ensure that the model performs well not only on training data but also on unseen data.

Overfitting

Overfitting occurs when a model memorizes training data instead of learning patterns.

Example

A student memorizes previous exam papers.

When asked the same questions:

- Score = 100%

When asked different questions:

- Score = 40%

This student is overfitted.

Characteristics:

- High training accuracy
 - Low testing accuracy
-

Underfitting

Underfitting occurs when a model fails to learn important patterns.

Example

A student studies only one chapter for a five-chapter exam.

Result:

- Poor performance everywhere

Characteristics:

- Low training accuracy
 - Low testing accuracy
-

Cross Validation

Cross Validation is a technique used to obtain a more reliable estimate of model performance.

K-Fold Cross Validation

Suppose $K = 5$.

Dataset divided into:

Fold 1
Fold 2
Fold 3
Fold 4
Fold 5

Process:

1. Train on folds 2,3,4,5 → Test on fold 1
2. Train on folds 1,3,4,5 → Test on fold 2
3. Continue until all folds are tested

The final accuracy is the average of all results.

Advantages:

- Better evaluation
- Reduces bias
- Useful for small datasets

CODE:-

```
from sklearn.model_selection import cross_val_score
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.datasets import load_iris

iris = load_iris()

model = DecisionTreeClassifier()

scores = cross_val_score(

    model,

    iris.data,

    iris.target,

    cv=5

)

print(scores)

print("Average Accuracy:", scores.mean())
```

2. Regression Techniques

Regression is a supervised learning technique used to predict continuous numerical values.

Examples:

- House prices
- Temperature
- Fuel consumption
- Salary prediction
- Stock prices

The output is always a number.

Linear Regression

Definition

Creates a straight line relationship between input and output.

Linear Regression is the simplest regression algorithm.

Idea

As one variable increases, another variable changes proportionally.

Examples:

- Experience increases → Salary increases
- Area increases → House price increases

The model tries to find the best straight line that fits the data.

Advantages

- Simple to understand
- Fast training
- Easy interpretation

Limitations

- Works only when relationships are approximately linear
- Cannot capture complex patterns

Real-World Applications

- House price prediction
- Sales forecasting
- Revenue estimation
- Salary prediction

CODE:-

```
import pandas as pd

from sklearn.linear_model import LinearRegression

data = {

    'Experience':[1,2,3,4,5],

    'Salary':[20000,30000,40000,50000,60000]

}

df = pd.DataFrame(data)

X = df[['Experience']]
```

```
y = df['Salary']

model = LinearRegression()

model.fit(X,y)

prediction = model.predict([[6]])

print("Salary Prediction:", prediction)

Salary Prediction: 70000
```

Polynomial Regression

Polynomial Regression is used when data follows a curved relationship.

Instead of fitting a straight line, it fits a curve.

Why Needed?

Many real-world problems are not linear.

Example:

Study Hours vs Marks

Hours	Marks
1	40
2	60
3	75
4	85
5	90

Marks increase rapidly at first and then gradually slow down.

A straight line cannot represent this accurately.

Polynomial Regression creates a curve that follows the trend better.

Applications

- Population growth
- Stock market trends
- Sales growth
- Manufacturing optimization

CODE:-

```
import numpy as np

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5]).reshape(-1,1)

y = np.array([40,60,75,85,90])

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

model = LinearRegression()

model.fit(X_poly,y)

prediction = model.predict(poly.transform([[6]]))

print(prediction)
```

3. Classification Techniques

Classification is a supervised learning technique used to predict categories or classes.

Output examples:

- Yes / No
- Spam / Not Spam
- Pass / Fail
- Disease / No Disease

Unlike regression, classification predicts labels instead of numbers.

Logistic Regression

Despite its name, Logistic Regression is mainly used for classification.

It predicts the probability that an observation belongs to a particular class.

Output ranges between:

0 to 1

Example:

0.95 → Disease Present

0.05 → Disease Absent

Applications

- Email spam detection
- Medical diagnosis
- Customer churn prediction
- Fraud detection

Advantages

- Simple
- Fast
- Works well for binary classification

Mini Project

Customer Purchase Prediction

Input:

- Age
- Salary

Output:

- Buy Product (Yes/No)

Limitations

- Cannot handle highly complex boundaries \

CODE:-

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split
```

```
data = load_breast_cancer()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    test_size=0.2,
    random_state=42
)

model = LogisticRegression(max_iter=5000)

model.fit(X_train,y_train)

print("Accuracy:", model.score(X_test,y_test))
```

K-Nearest Neighbors (KNN)

KNN is one of the simplest classification algorithms.

It classifies a new data point based on nearby examples.

Working Principle

1. Select K nearest neighbors.
2. Count class labels.
3. Majority class wins.

Example

Suppose:

Nearby Students:

```
Student A → Science
Student B → Science
Student C → Science
Student D → Commerce
Student E → Science
```

New student most likely belongs to:

Science

because the majority belongs to Science.

Advantages

- Easy to understand
- No training phase
- Good for small datasets

Limitations

- Slow on large datasets
- Sensitive to irrelevant features

Applications

- Recommendation systems
- Image recognition
- Medical diagnosis
- Pattern recognition

CODE:-

```
from sklearn.neighbors import KNeighborsClassifier

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(

    iris.data,

    iris.target,

    test_size=0.2,

    random_state=42

)

model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train,y_train)

print("Accuracy:", model.score(X_test,y_test))
```

Mini Project

Fruit Classification

Input:

- Weight
- Color

Output:

- Apple / Orange
-

4. Tree-Based Models

Tree-based models make decisions using a hierarchical structure similar to a flowchart.

These models ask a sequence of questions and arrive at a final prediction.

Humans naturally make decisions in a tree-like manner.

Example:

```
Is weather sunny?  
  |  
  Yes  
  |  
Go outside
```

Decision Tree

Decision Tree is a supervised learning algorithm used for both classification and regression.

It consists of:

Root Node

Starting point of the tree.

Internal Node

Decision point.

Branch

Possible outcomes.

Leaf Node

Final prediction.

Example

Loan Approval System

```
Age > 18?  
  |  
  Yes  
  |  
Salary > 30000?  
  |  
  Yes  
  |  
Loan Approved
```

Advantages

- Easy to visualize
- Easy to interpret
- Handles non-linear data

Disadvantages

- Can overfit
- Sensitive to small changes in data

Applications

- Credit approval
- Medical diagnosis
- Customer segmentation
- Risk analysis

CODE:-

```
from sklearn.tree import DecisionTreeClassifier  
  
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split  
  
iris = load_iris()  
  
X_train, X_test, y_train, y_test = train_test_split(  
    iris.data,  
    iris.target,
```

```
test_size=0.2,  
  
random_state=42  
  
)  
  
model = DecisionTreeClassifier()  
  
model.fit(X_train,y_train)  
  
print("Accuracy:", model.score(X_test,y_test))
```

Model Intuition

Model intuition means understanding why a model makes a particular prediction.

Many beginners focus only on accuracy.

However, understanding the reasoning behind predictions is equally important.

Example

House Price Prediction

Features:

- Area
- Location
- Bedrooms

Model learns:

Larger Area → Higher Price

Better Location → Higher Price

More Bedrooms → Higher Price

The model identifies relationships between inputs and outputs.

Why Model Intuition Matters?

It helps:

- Explain predictions
- Build trust
- Detect bias
- Improve model performance

- Debug errors

Complete Real-World Project for Students

House Price Prediction

Dataset Features

Area Bedrooms Age

1000 2 5

1500 3 2

2000 4 1

Target:

House Price

Steps

1. Load Dataset
2. Data Cleaning
3. Train-Test Split
4. Train Linear Regression
5. Predict House Price
6. Evaluate Accuracy

Skills Covered:

- Pandas
- Data Preparation
- Train-Test Split
- Linear Regression
- Model Evaluation